PAWRADISE

SUPERVISÉE PAR:
GHASSEN HAMOUDA



Plan

I IDÉE GÉNÉRALE ET PROBLÉMATIQUE

II PRINCIPALES FONCTIONNALITÉS

III ARCHITECTURE GLOBALE

IV TECHNOLOGIES CHOISIES

V MICROSERVICES ET BASES DE DONNÉES



MEMBRES DU PROJET



Eya gobji



Hiba Ibn Hadj Mohammed



Mayssem Jihene Rezgui



Mariem Abbes



Yosra Bejaoui

I

IDÉE GÉNÉRALE ET PROBLÉMATIQUE



Idée générale

PawGuardian est une plateforme unique qui réunit la protection, le sauvetage, l’adoption, la santé et le bien-être des animaux. Elle met en relation citoyens, refuges, bénévoles, vétérinaires et donateurs.



Problématique

Comment coordonner citoyens, sauveteurs, vétérinaires et associations pour sauver, soigner et placer plus vite les animaux en détresse ?

Acteurs dispersés

Citoyens, sauveteurs, vétérinaires et refuges ne sont pas connectés

Information perdue

Citoyens, sauveteurs, vétérinaires et refuges ne sont pas connectés

Réaction trop lente

Un animal blessé attend une intervention

II

PRINCIPALES FONCTIONNALITÉS



Principales fonctionnalités

1 Lost & Found

✓ Signalements perdus / trouvés
✓ Recherche par proximité
✓ Rapprochements réclamations

1 Animal Rescue

✓ Demandes classées par urgence
✓ Missions, équipes, bénévoles
✓ Suivi et rapport de mission

3 Adoption

✓ Annonces et candidatures
✓ Évaluation des adoptants
✓ Contrats et suivi

4 Marketplace

✓ Catalogue et panier
✓ Commandes et paiement
✓ Livraison et retours

5 VetCare

✓ Cliniques et rendez-vous
✓ Dossiers médicaux
✓ Vaccins et traitements

6 Dons & Ressources

✓ Campagnes et dons
✓ Stock de ressources
✓ Attribution aux refuges

III

ARCHITECTURE GLOBALE



architechture en microservices

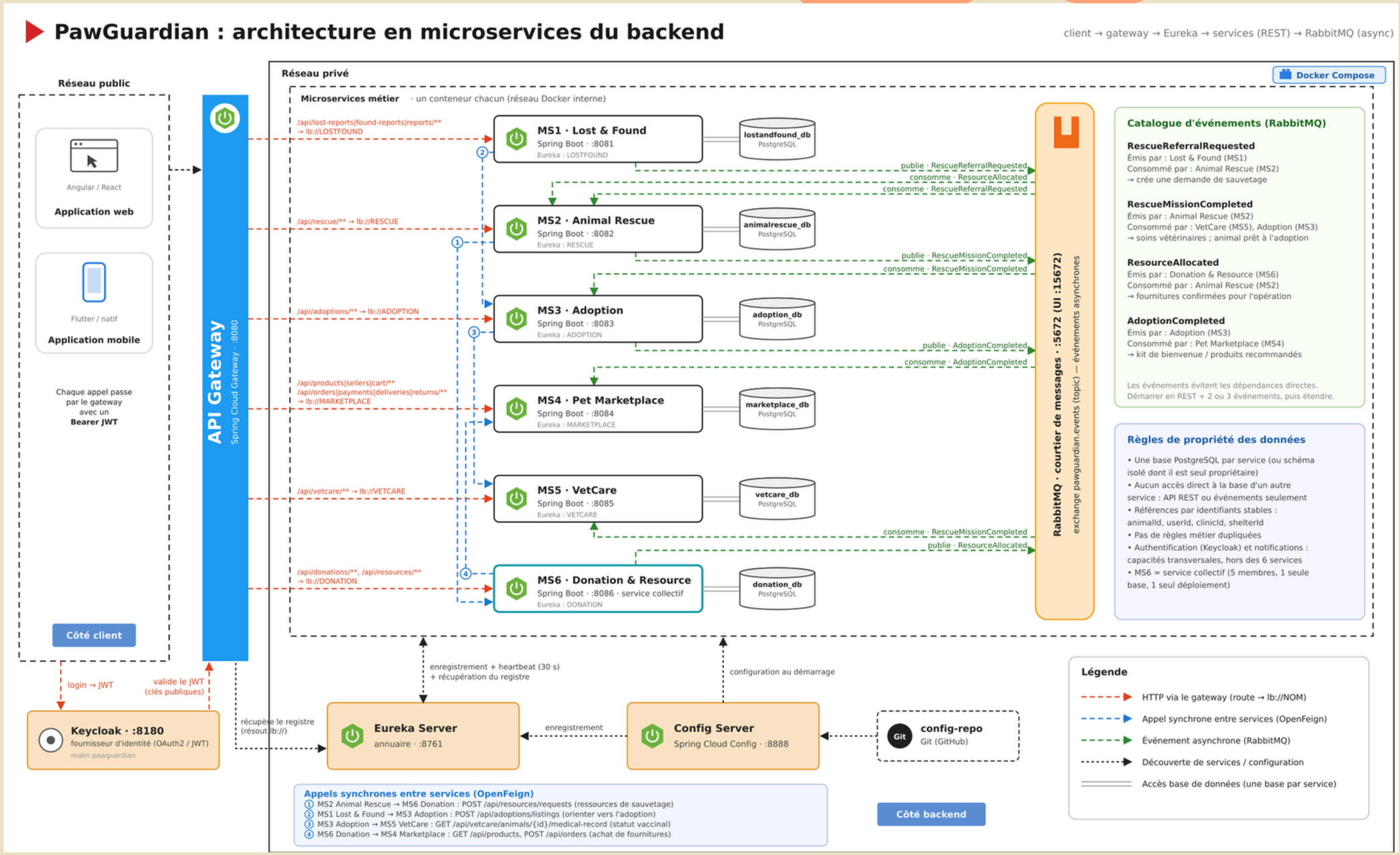


Diagramme de cas d'utilisation

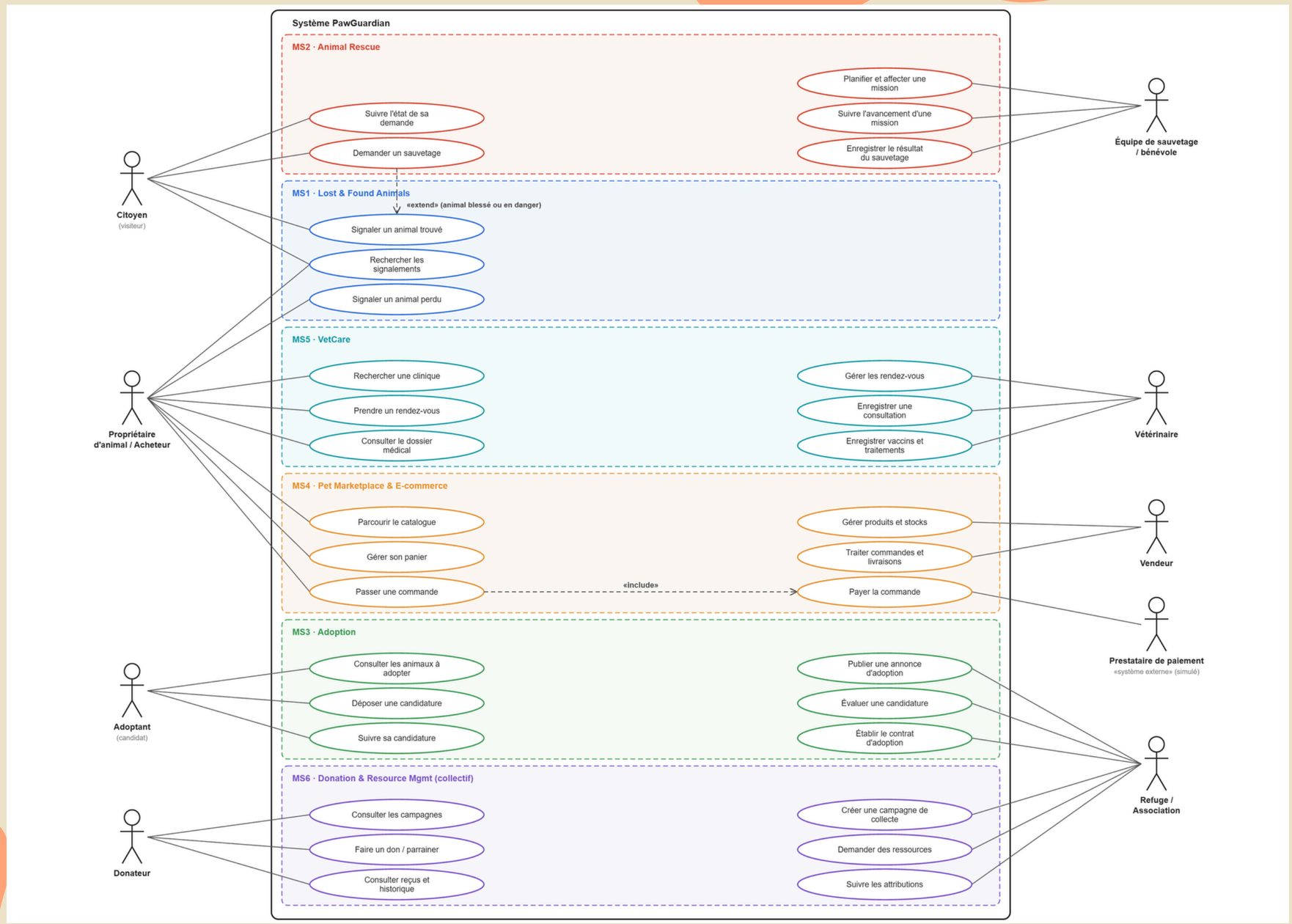


Diagramme de classe globale

Diagramme de classes global : Pawradise

6 microservices, une couleur par service. Animal Rescue au centre. Les liens entre services passent par des identifiants.

SYNCHRONE · REST

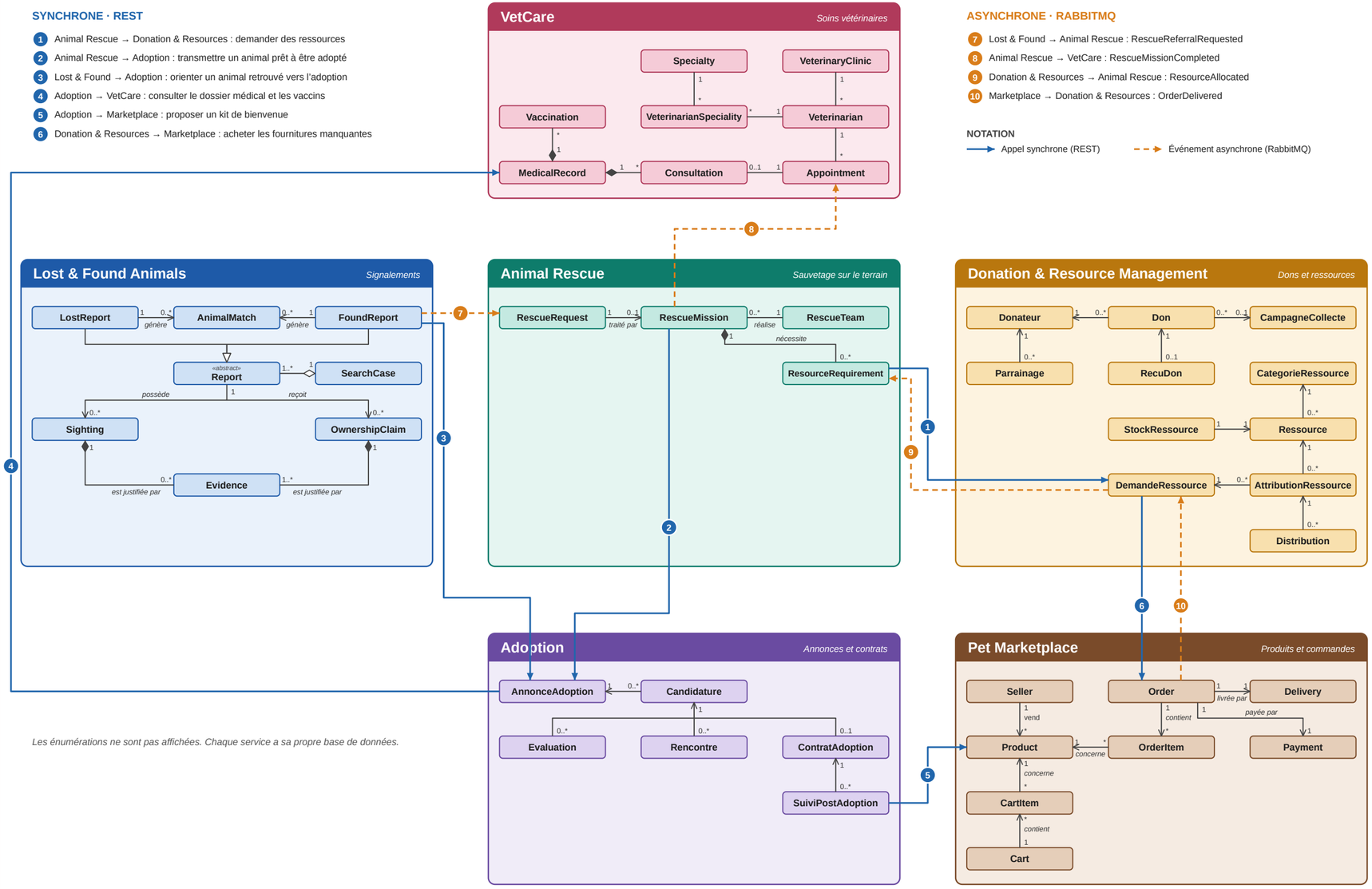
1. Animal Rescue → Donation & Resources : demander des ressources
2. Animal Rescue → Adoption : transmettre un animal prêt à être adopté
3. Lost & Found → Adoption : orienter un animal retrouvé vers l'adoption
4. Adoption → VetCare : consulter le dossier médical et les vaccins
5. Adoption → Marketplace : proposer un kit de bienvenue
6. Donation & Resources → Marketplace : acheter les fournitures manquantes

ASYNCHRONE · RABBITMQ

7. Lost & Found → Animal Rescue : RescueReferralRequested
8. Animal Rescue → VetCare : RescueMissionCompleted
9. Donation & Resources → Animal Rescue : ResourceAllocated
10. Marketplace → Donation & Resources : OrderDelivered

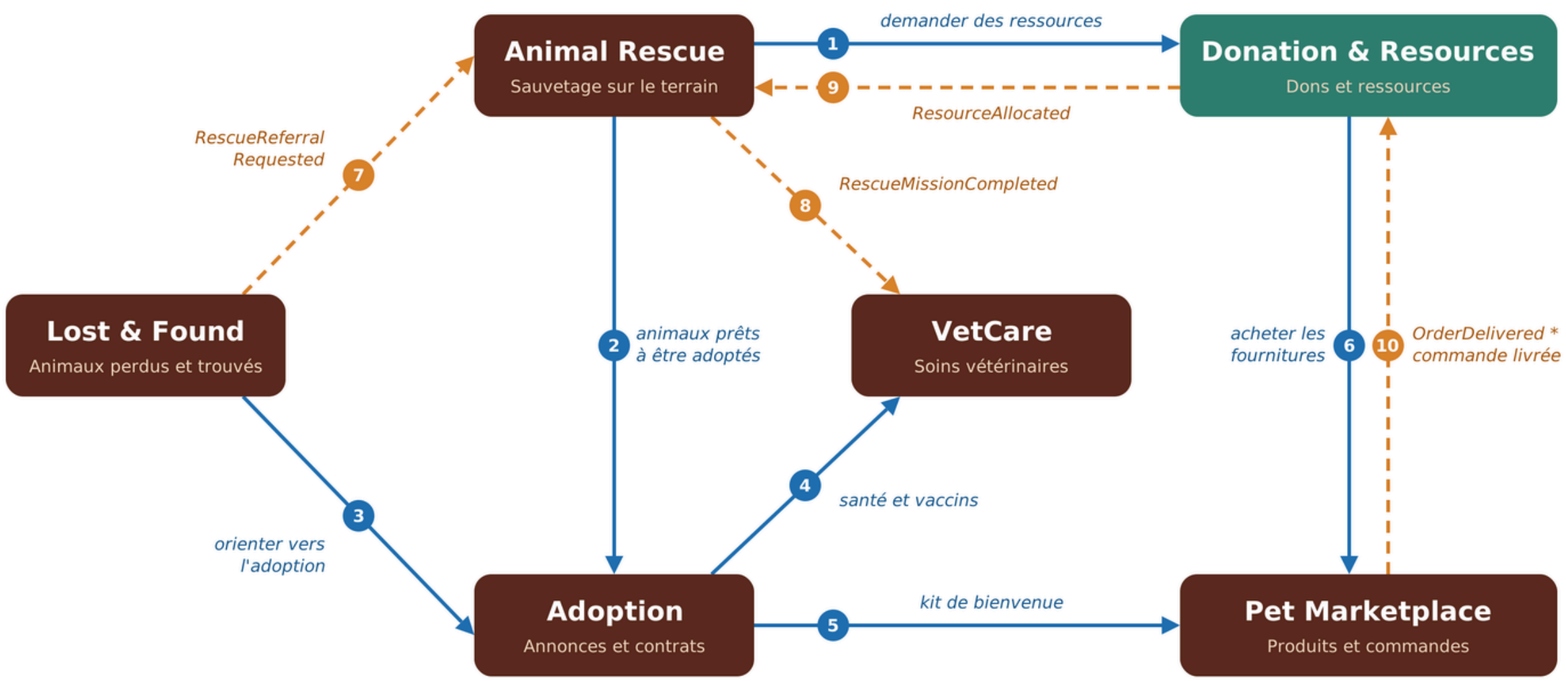
NOTATION

Appel synchrone (REST) — Événement asynchrone (RabbitMQ)



Les énumérations ne sont pas affichées. Chaque service a sa propre base de données.

Carte des contextes



Trait plein : appel synchrone (REST). L'appelant interroge le service appelé.

Pointillé : événement asynchrone (RabbitMQ). Le publieur prévient l'abonné, sans attendre.

Service individuel Service collectif (toute l'équipe)

** Événement proposé par le responsable du Marketplace, à valider avec l'équipe (absent du catalogue d'événements actuel).*

LES ÉCHANGES ENTRE LES SERVICES

SYNCHRONE · REST (OPENFEIGN)

1. **Animal Rescue → Donation & Resources**
   demander nourriture, médicaments, équipements
2. **Animal Rescue → Adoption**
   transmettre un animal prêt à être adopté
3. **Lost & Found → Adoption**
   orienter un animal retrouvé vers l'adoption
4. **Adoption → VetCare**
   consulter le dossier médical et les vaccins
5. **Adoption → Marketplace**
   proposer un kit de bienvenue
6. **Donation & Resources → Marketplace**
   acheter les fournitures manquantes au stock

ASYNCHRONE · RABBITMQ

7. **Lost & Found → Animal Rescue**
   RescueReferralRequested : animal blessé ou en danger
8. **Animal Rescue → VetCare**
   RescueMissionCompleted : orientation vers un vétérinaire
9. **Donation & Resources → Animal Rescue**
   ResourceAllocated : fournitures confirmées
10. **Marketplace → Donation & Resources**
    OrderDelivered * : commande livrée

Adoption → services concernés
AdoptionCompleted : consommateurs à définir

IV

TECHNOLOGIES CHOISIES



Technologies choisies

Backend



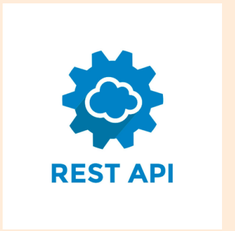




Base de Données

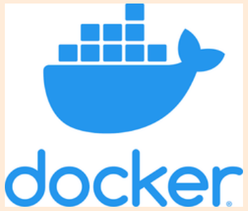


Communication





Infrastructure



API Gateway

Eureka

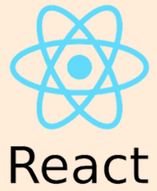
Sécurité





Frontend





V

MICROSERVICES ET BASES DE DONNÉES



Organisation du dépot git et gestion du projet :

Structure du dépôt et responsables

pawguardian-<classe>-<groupe>/

├── **Backend/**	
│ ├── lost-found-service/	Hiba
│ ├── animal-rescue-service/	Mariem
│ ├── adoption-service/	Eya
│ ├── marketplace-service/	Yosra
│ ├── vetcare-service/	Mayssem
│ └── donation-resource-service/	Toute l'équipe
├── **Frontend/**	
├── **Documentation/**	
│ ├── Presentation/	
│ ├── Diagrams/	
│ └── Dossier technique	
└── **README.md**	

Règles de travail Git

- main : version stable. develop : intégration des services
- Une branche par fonctionnalité, par exemple feature/adoption-candidatures
- Service collectif : une branche par membre et par partie, par exemple feature/donation-campagnes
- Fusion dans develop par pull request relue par un autre membre

GitHub Projects

- Colonnes : À faire, En cours, En revue, Terminé
- Une issue par workflow ou endpoint, par exemple « POST /api/adoptions/applications »
- Un label par service pour filtrer le travail de chacun
- Jalon : validation intermédiaire (séance 4)

Répartition des taches :

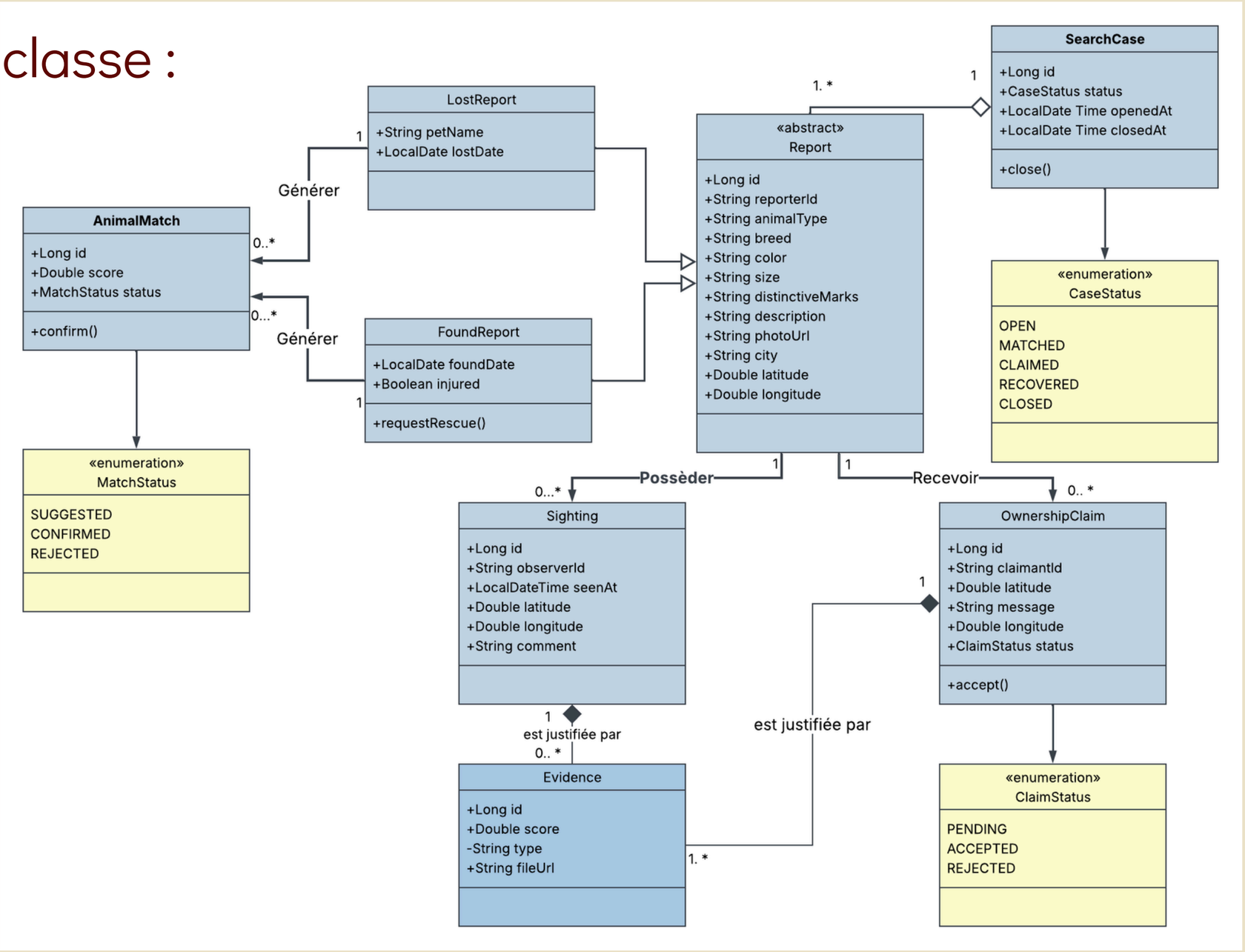
Membre	Service individuel	Part du Service collectif: Donation&Resource Management
Hiba Ibn Hadj Mohamed	Lost & Found animals	Campagnes de collecte, enregistrement des dons et parcours des donateurs
Mariem Abbes	Animal Rescue	Catalogue des ressources matérielles, catégories et stock
Eya Gobji	Adoption	Demandes de ressources, validation et workflows d’attribution
Yosra Bejaoui	Pet Marketplace & E-commerce	Parrainages, reçus et rapports de campagne
Mayssem Jihene Rezgui	VetCare	Intégration, coordination des API, tests et appui au déploiement

Micro Service I

Lost & Found Animals



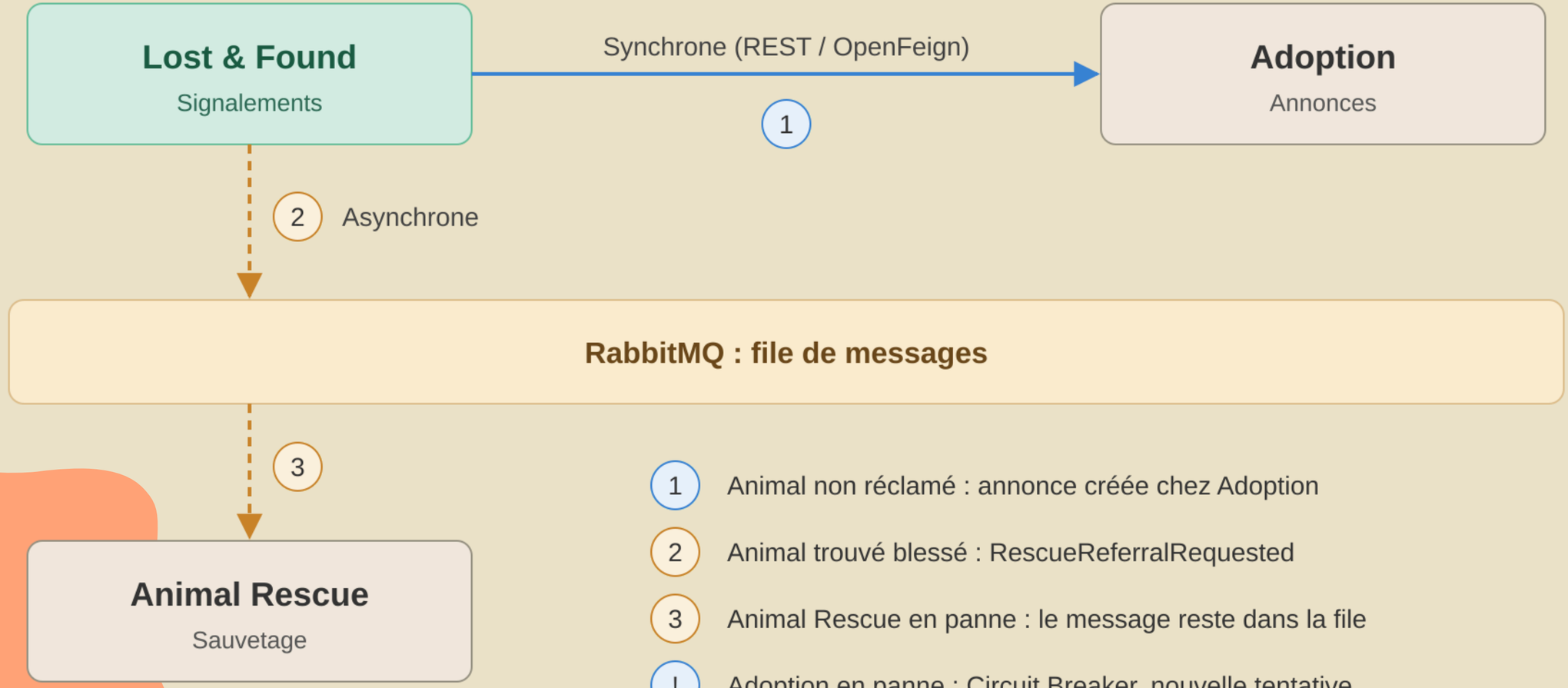
diagramme de classe :



API REST du Animal Rescue :

Méthode	Endpoint	Codes de réponse
POST	/api/lost-reports	201, 400, 401
POST	/api/found-reports	201, 400, 401
GET	/api/reports/search	200, 400
GET	/api/reports/nearby	200, 400
POST	/api/reports/{id}/sightings	201, 400, 404
POST	/api/reports/{id}/claims	201, 400, 404, 409
GET	/api/matches/{reportId}	204, 404
PUT	/api/reports/{id}/status	200, 400, 403, 404, 409

Interactions avec les autres microservices :



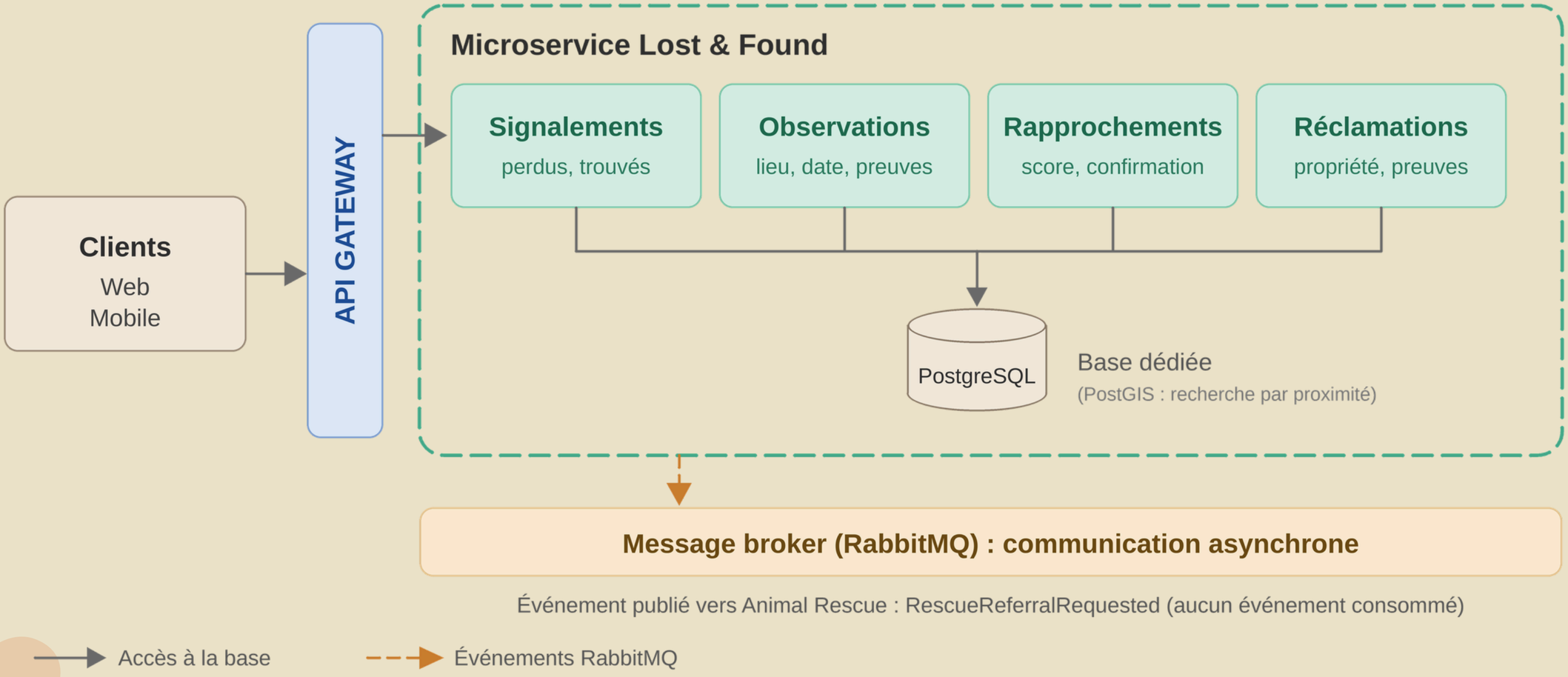
1 Animal non réclamé : annonce créée chez Adoption

2 Animal trouvé blessé : RescueReferralRequested

3 Animal Rescue en panne : le message reste dans la file

! Adoption en panne : Circuit Breaker, nouvelle tentative

Architecture de microservice :



Micro Service II

Animal rescue

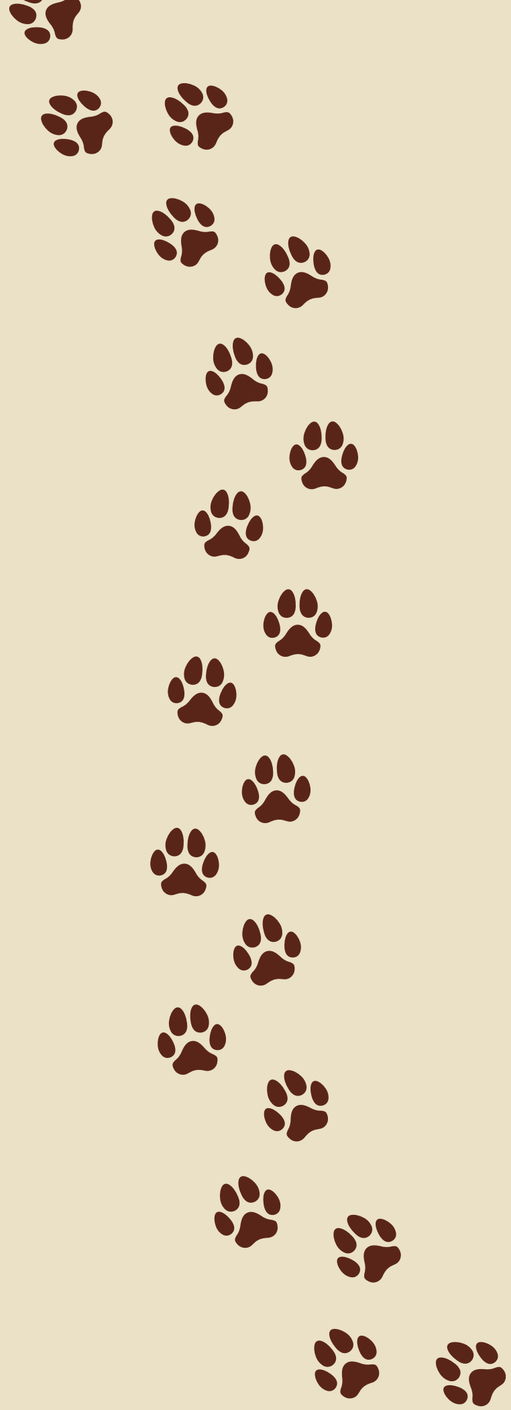
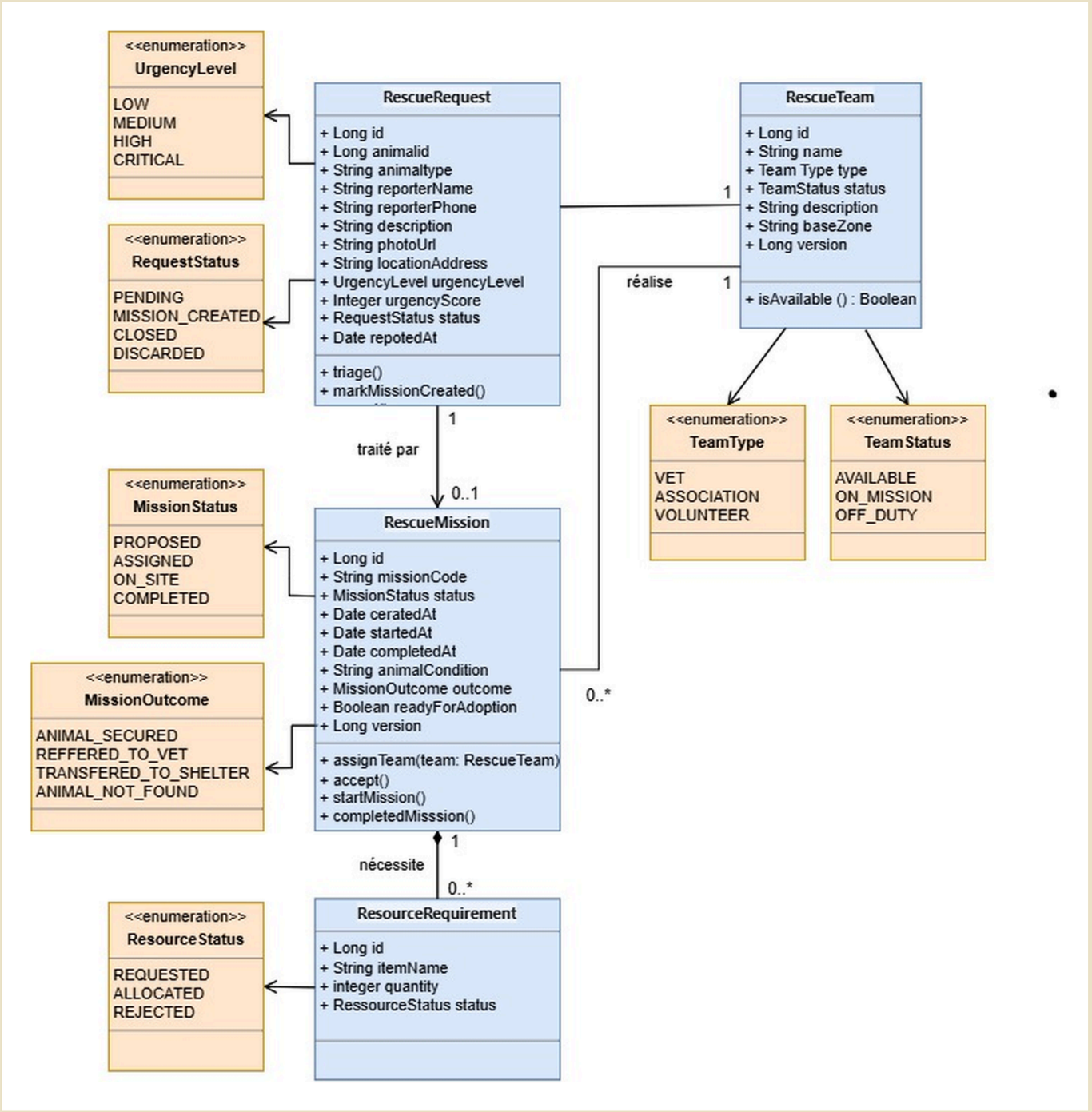


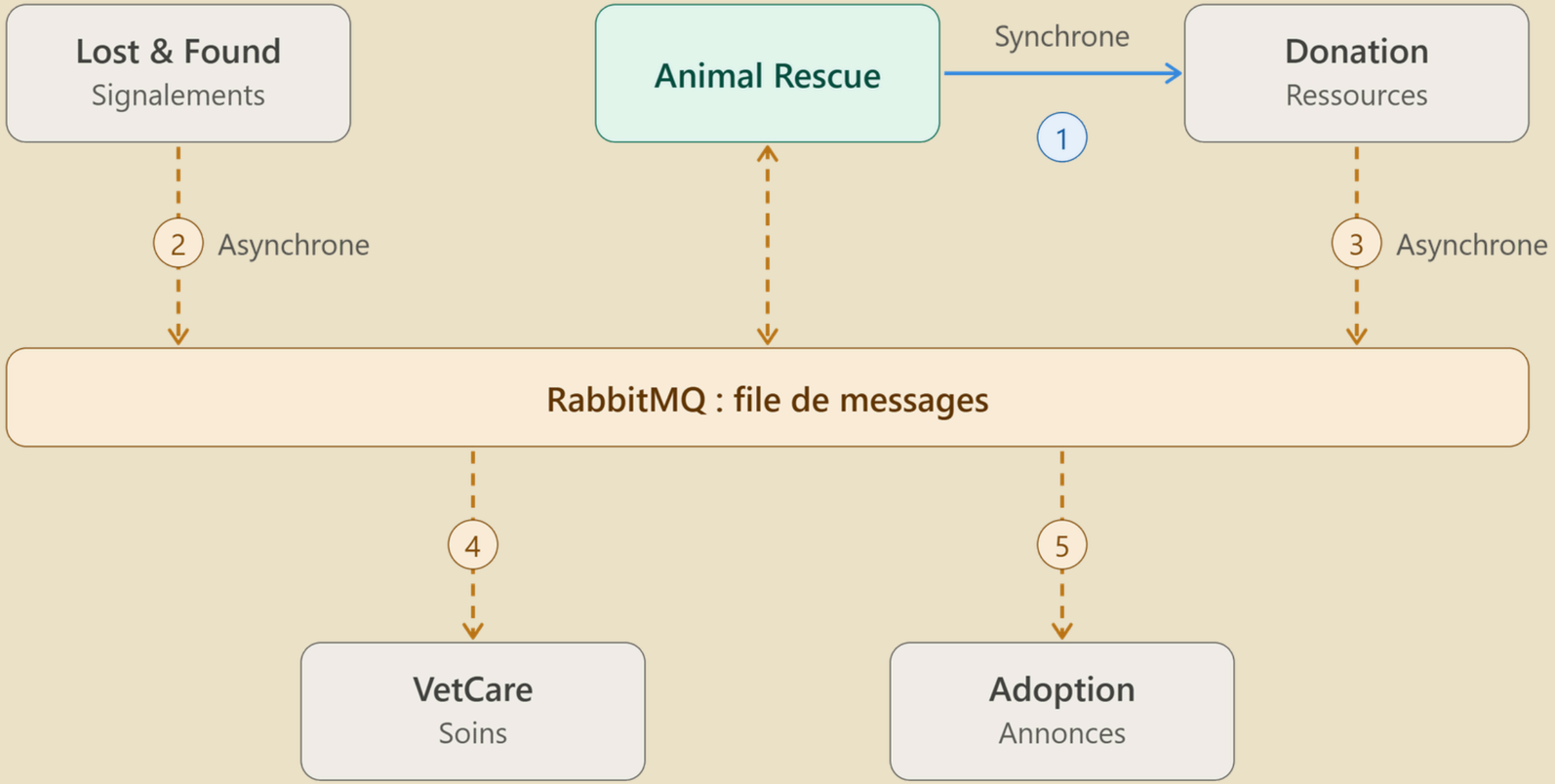
diagramme de classe :



API REST du Animal Rescue :

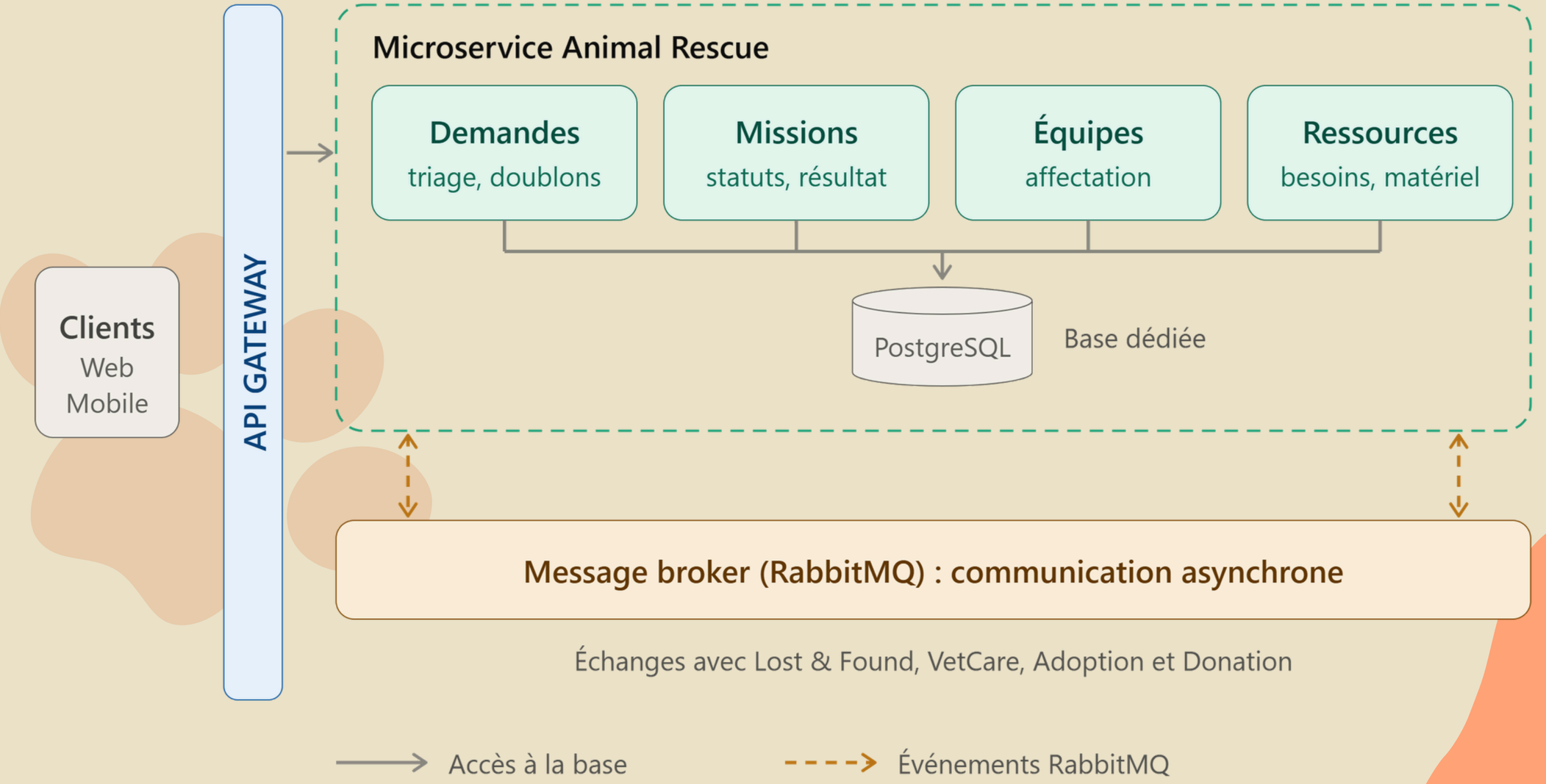
Méthode	Endpoint	Codes de réponse
POST	/api/rescue/requests	201, 200, 400
GET	/api/rescue/requests	200
POST	/api/rescue/missions	201, 400, 409
POST	/api/rescue/missions/{id}/assign	200, 404, 409
POST	/api/rescue/missions/{id}/accept	200, 404, 409
PUT	/api/rescue/missions/{id}/status	200, 400, 409
POST	/api/rescue/missions/{id}/resources	201, 404, 503

Interactions avec les autres microservices :



1. Animal Rescue demande des ressources à Donation (REST)
2. Animal blessé signalé : RescueReferralRequested
3. Ressources confirmées : ResourceAllocated
4. Mission terminée : RescueMissionCompleted vers VetCare
5. Animal prêt : AnimalReadyForAdoption vers Adoption

Architecture de microservice :



Micro Service III

Adoption

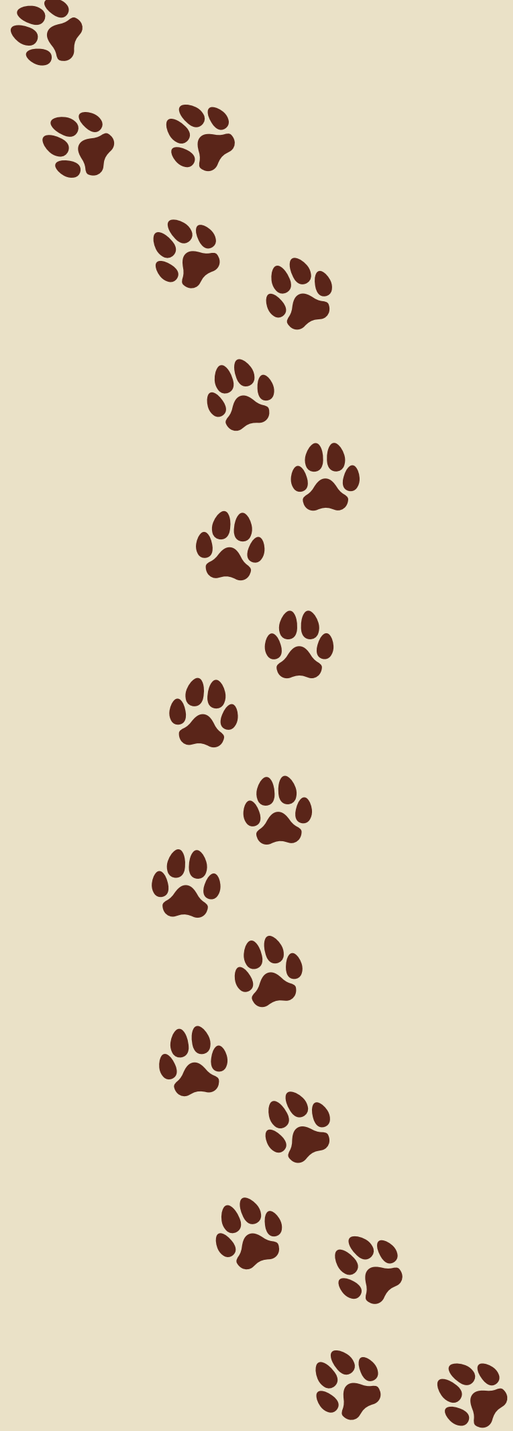
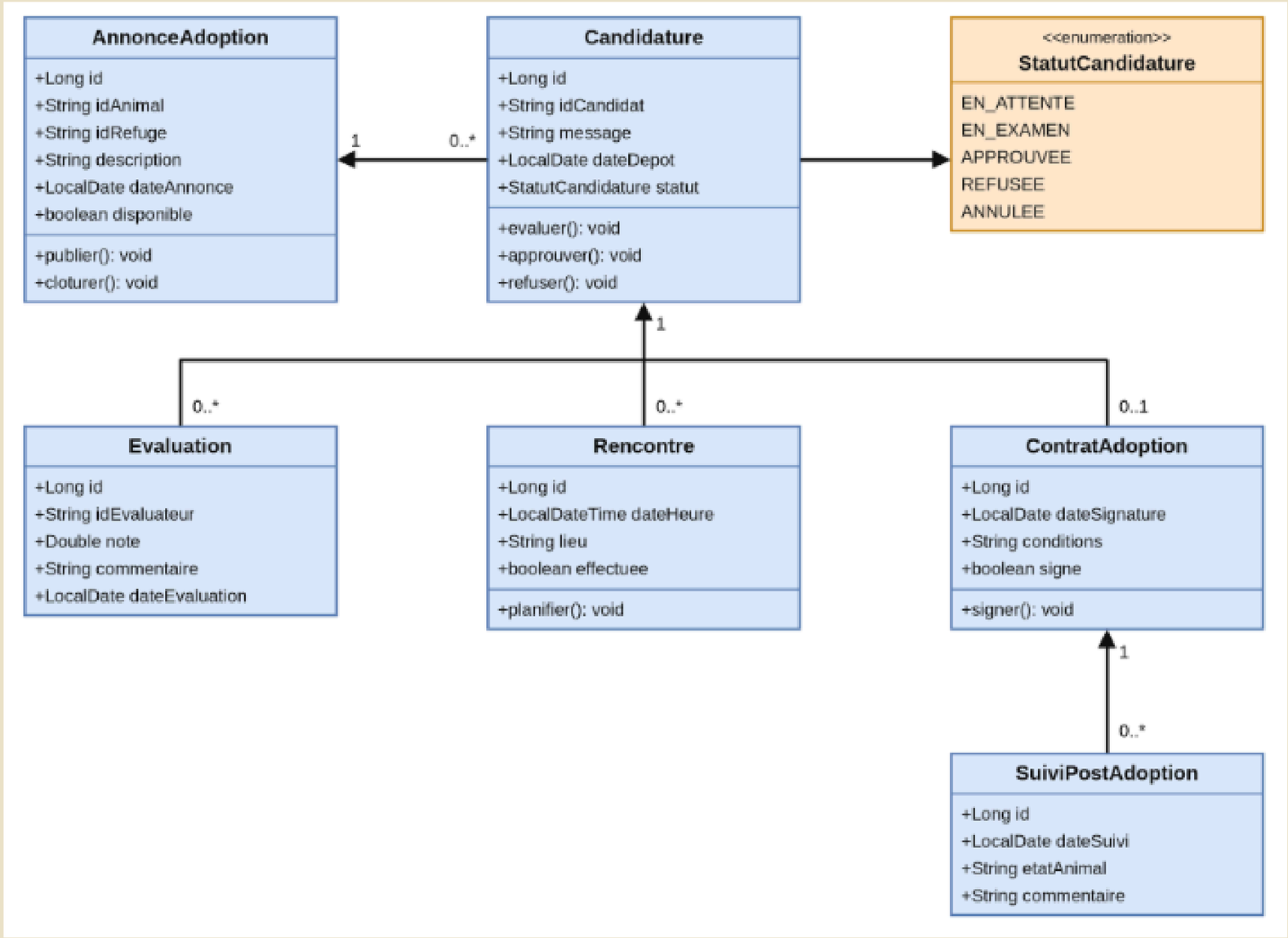


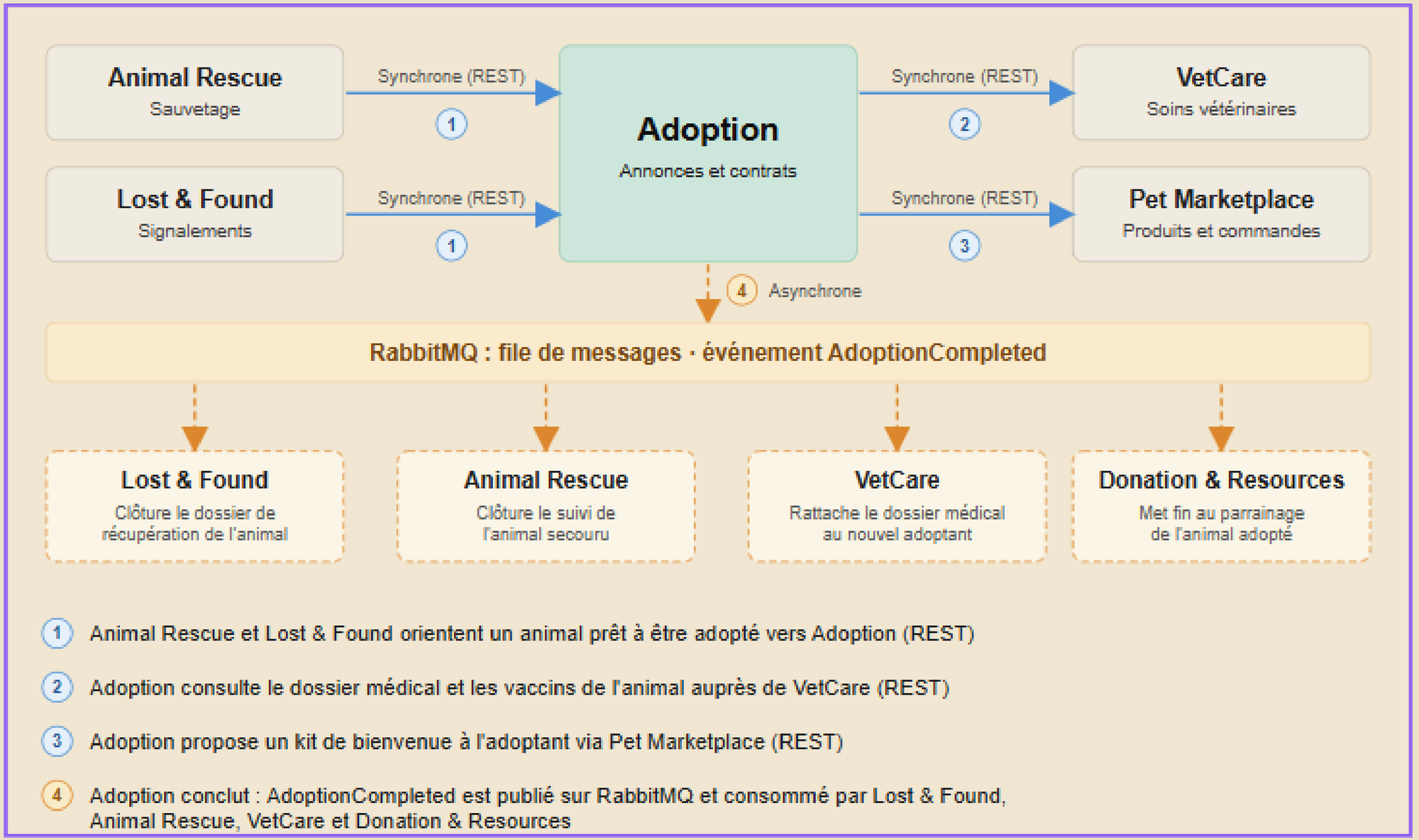
Diagramme de classe :



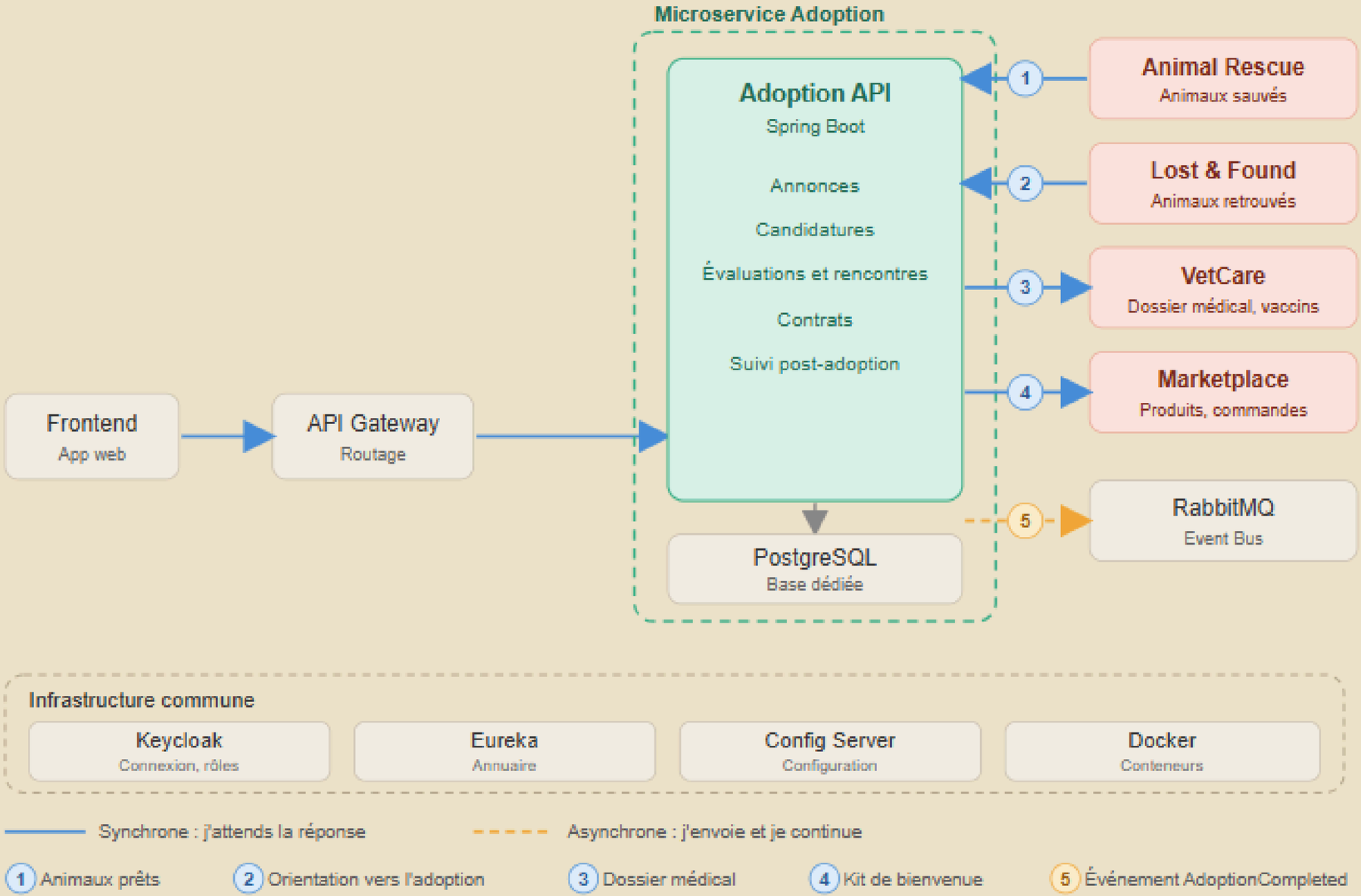
II API REST du Service Adoption :

Méthode	Endpoint	Codes de réponse
POST	/api/adoptions/listings	201, 400
GET	/api/adoptions/available	200
POST	/api/adoptions/applications	201, 400, 404, 409
GET	/api/adoptions/applications/{id}	200, 404
POST	/api/adoptions/applications/{id}/evaluate	201, 400, 404, 409
PUT	/api/adoptions/applications/{id}/decision	200, 400, 404, 409
POST	/api/adoptions/contracts	201, 400, 404, 409
GET	/api/adoptions/history	200

III Interactions avec les autres microservices :



Architecture de microservice :



Micro Service IV:
Marketplace

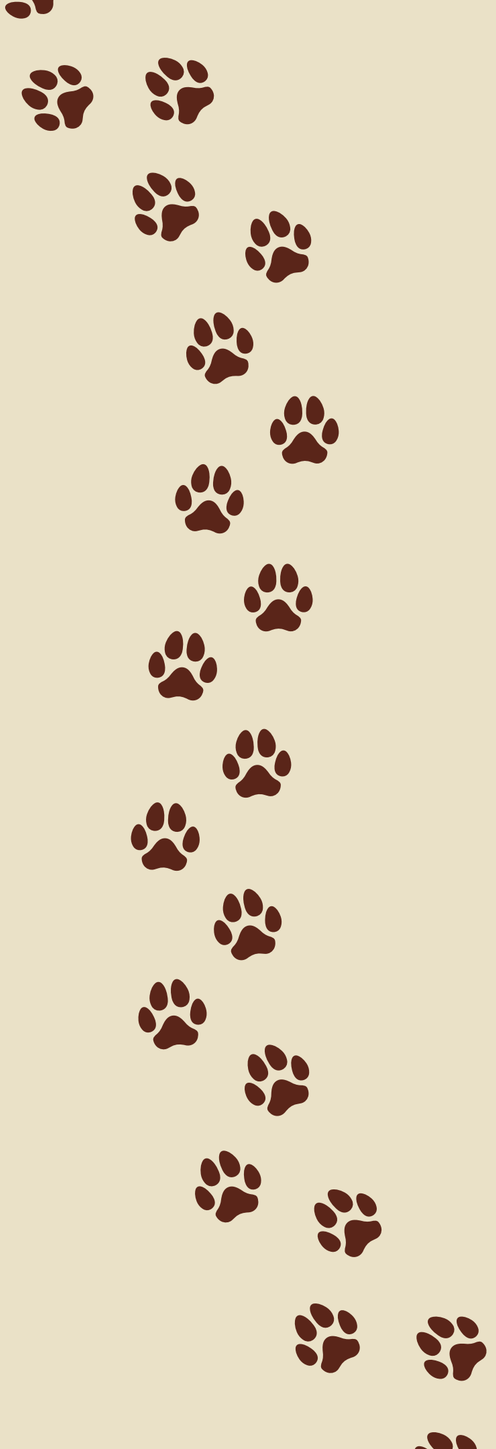
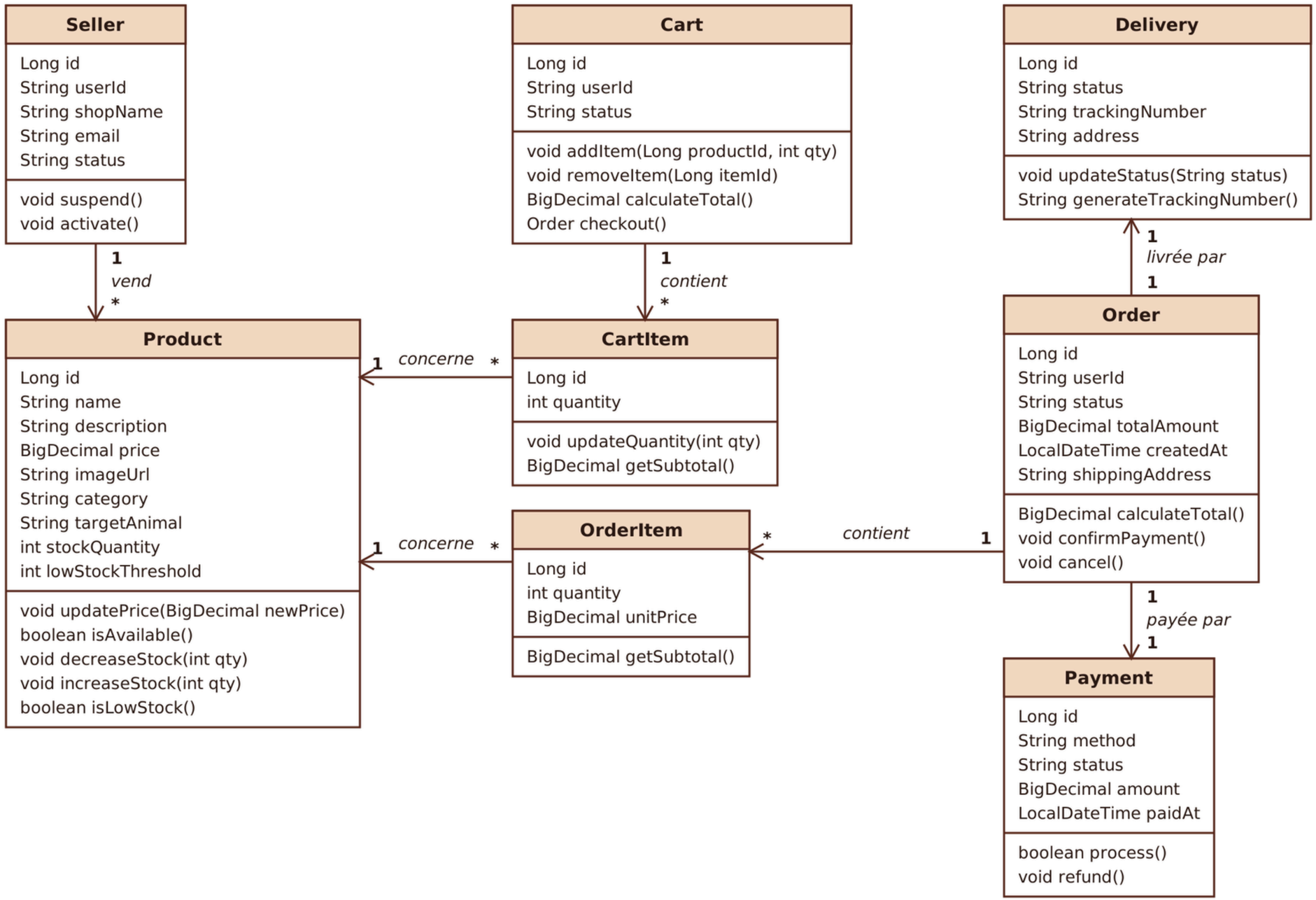


Diagramme de classes : microservice Pet Marketplace (Pawradise)

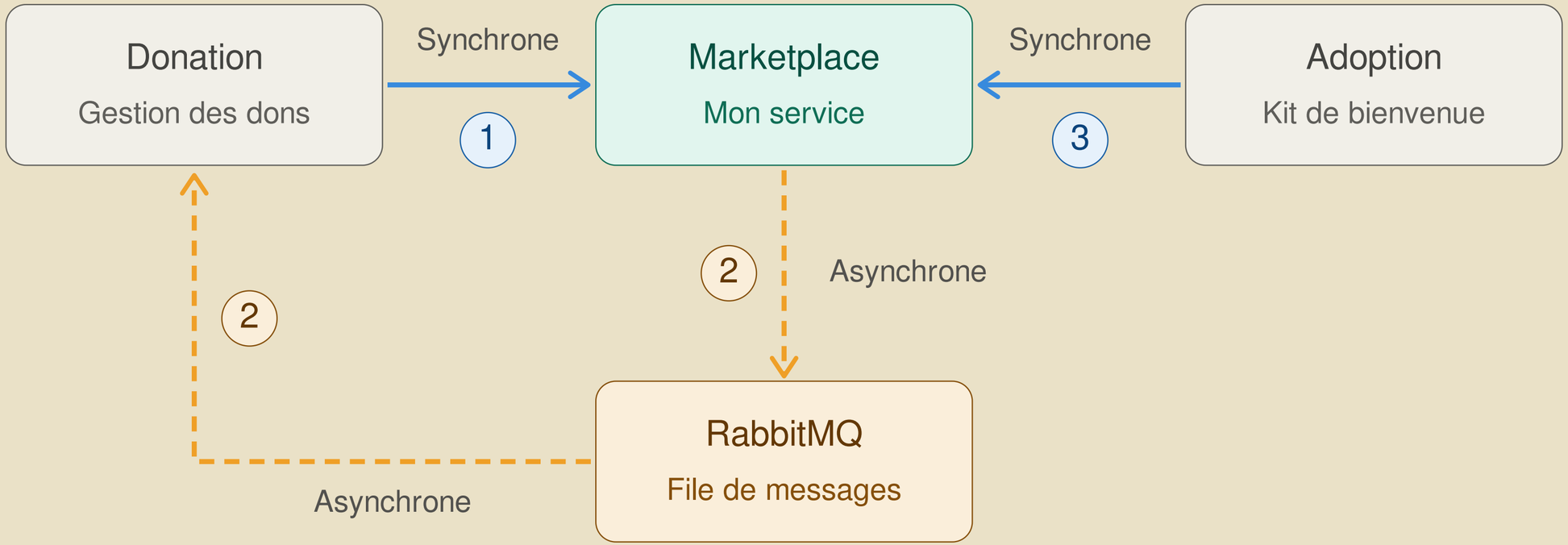


API REST du Marketplace :

ÉVÉNEMENT PUBLIÉ : ORDERDELIVERED

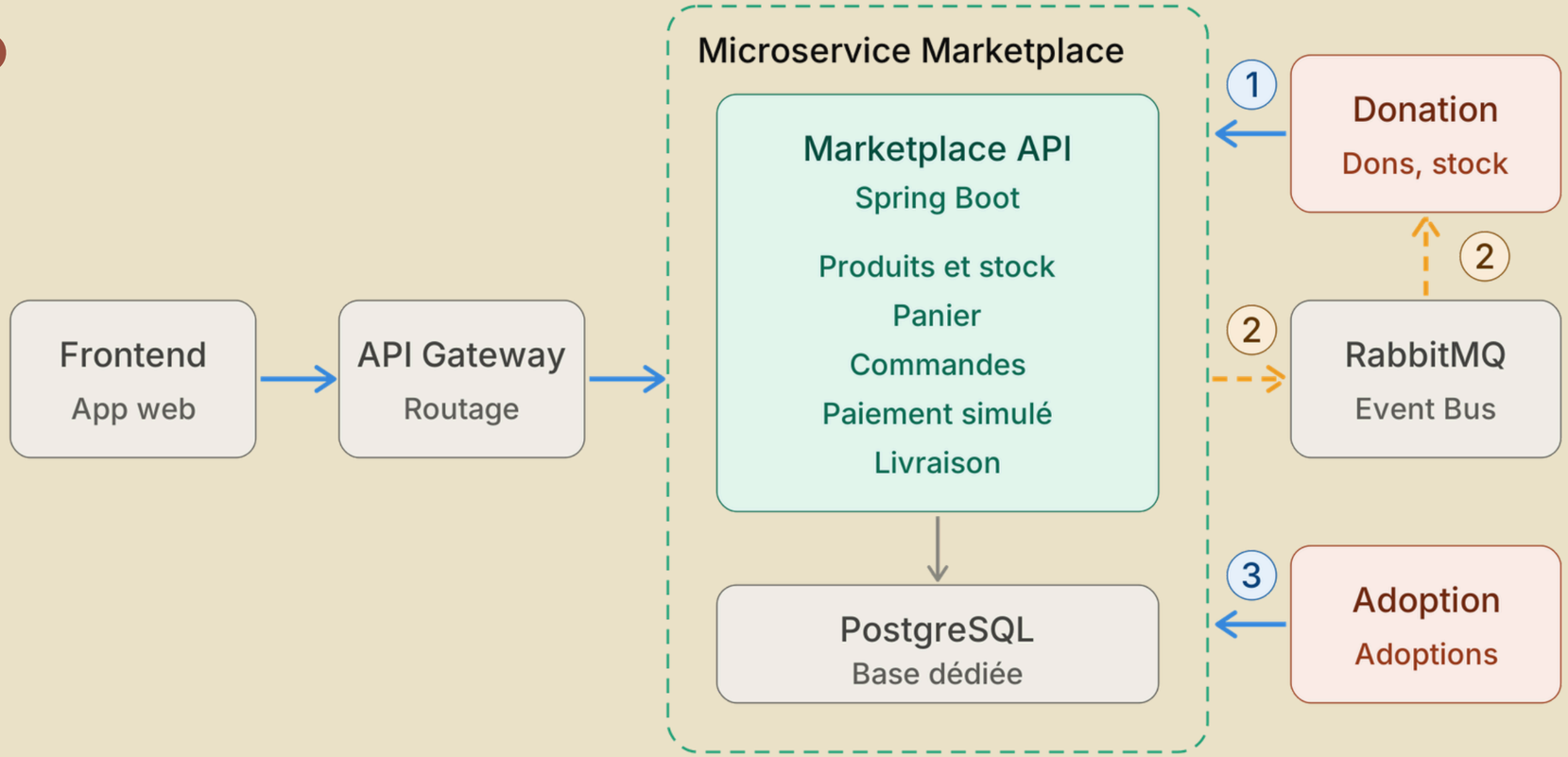
Méthode	Endpoint	Codes de réponse
GET	/api/products	200
POST	/api/cart/items	201,404,409
POST	/api/orders	201,400,409
POST	/api/payments	201,400,404
GET	/api/deliveries/{id}	200,404

Interactions avec les autres microservices :



1. Donation cherche un produit et commande
2. Commande livrée : message OrderDelivered envoyé
3. Adoption affiche des produits (kit de bienvenue)

Architecture de microservice :



Micro Service v :

VetCare

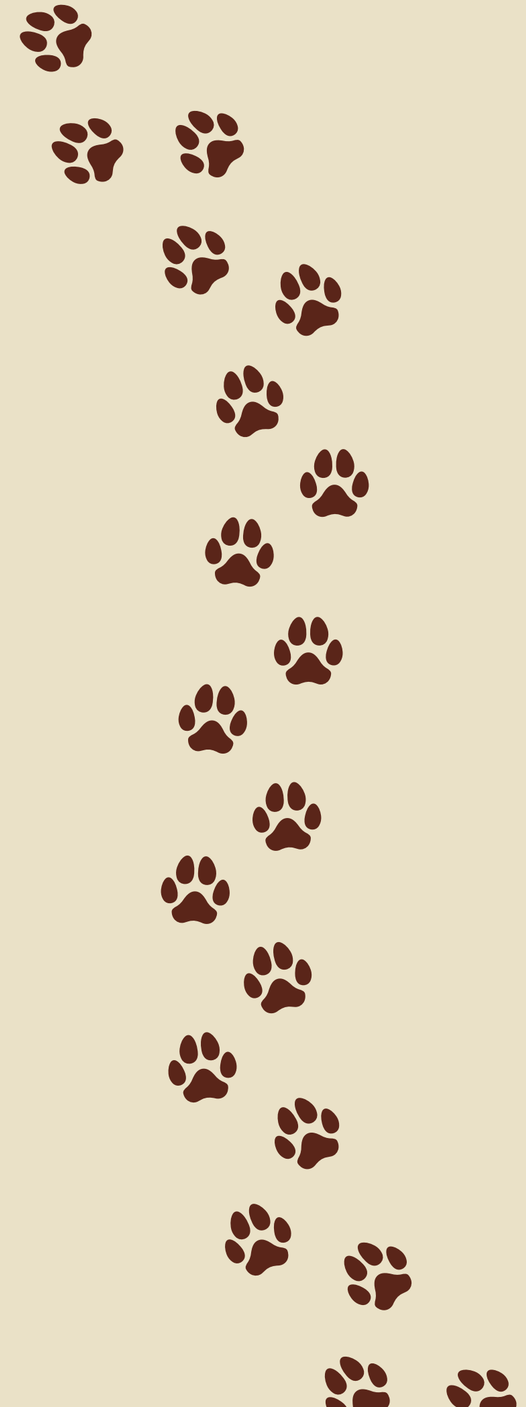
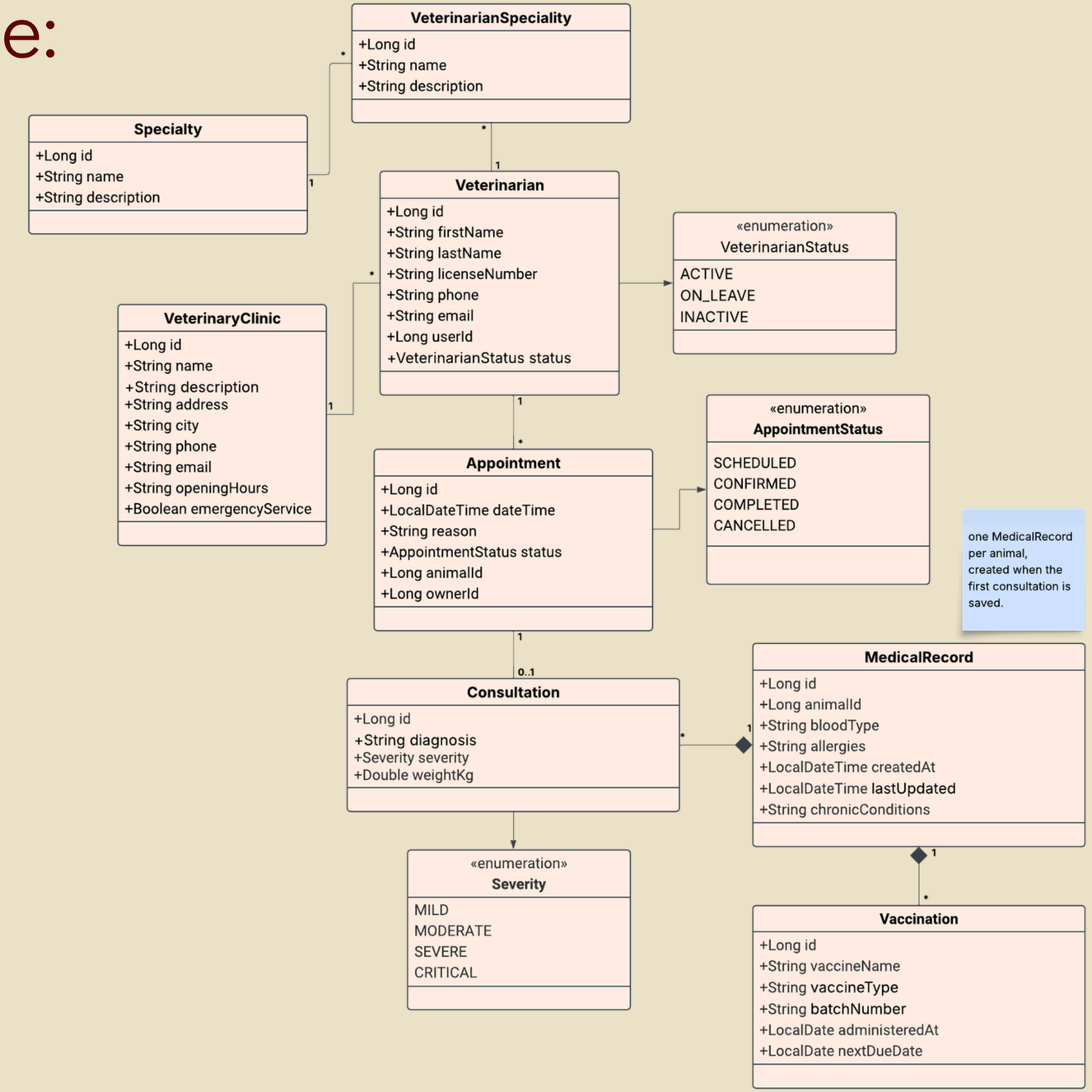


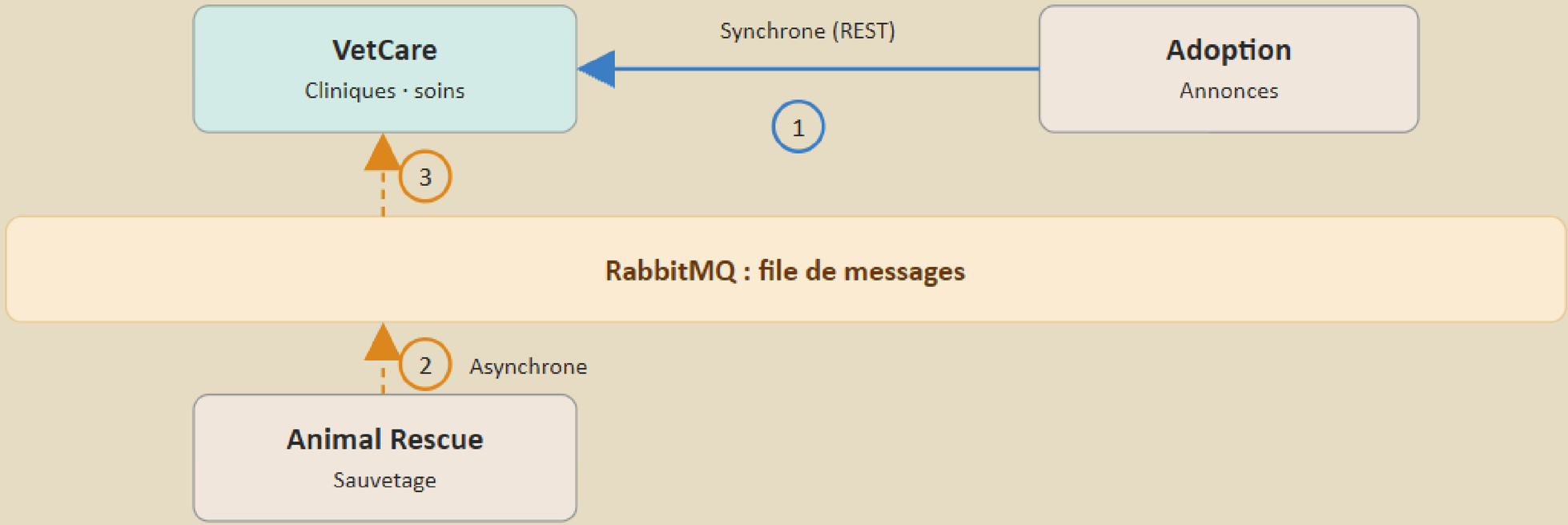
diagramme de classe:



II API REST du VetCare :

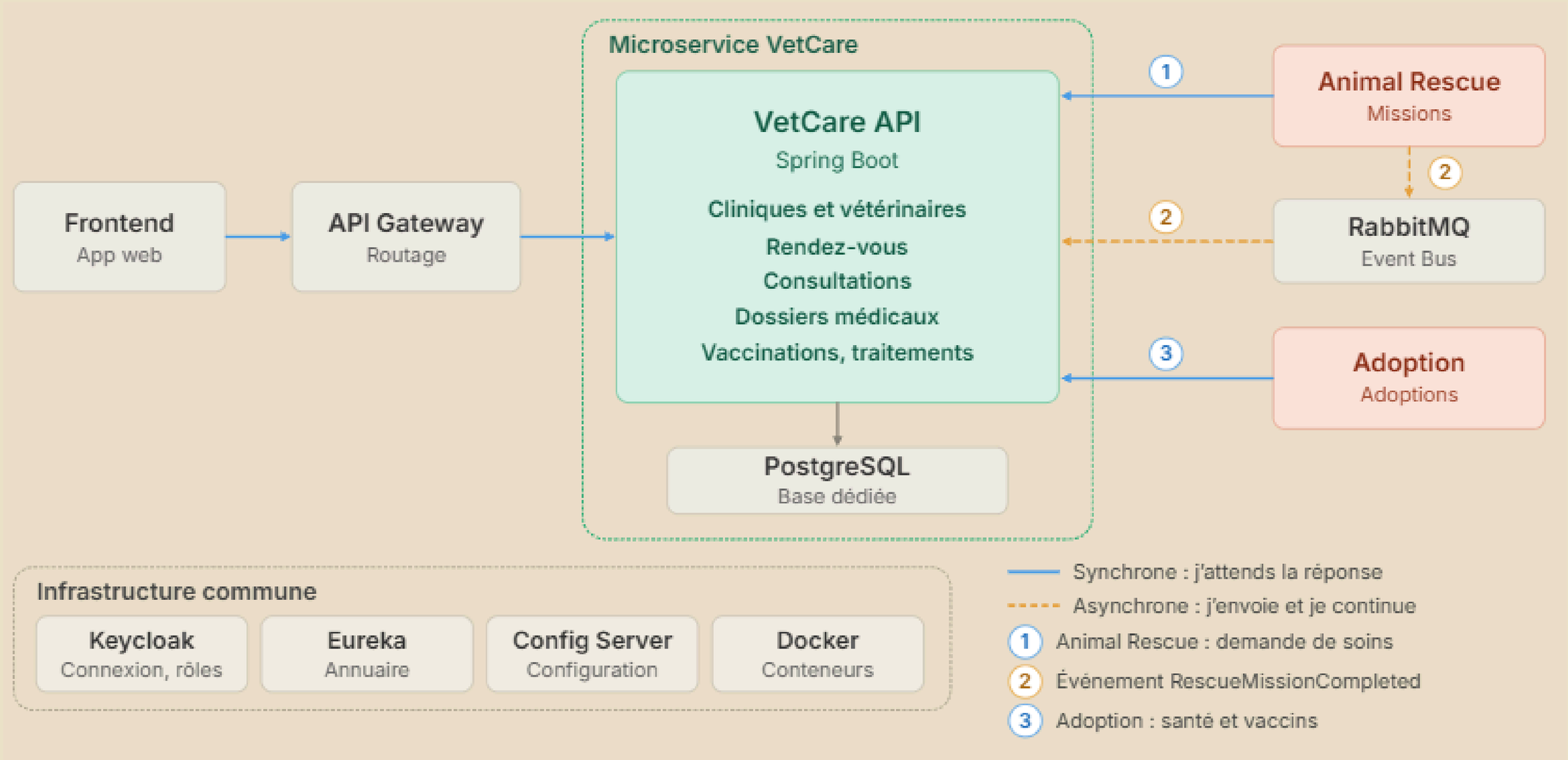
Méthode	Endpoint	Codes de réponse
POST	/api/vetcare/clinics	201, 400, 409
POST	/api/vetcare/veterinarians	201, 400, 404
GET	/api/vetcare/clinics/search	200
POST	/api/vetcare/appointments	201, 400, 404, 409
GET	/api/vetcare/animals/{id}/medical-record	200, 404
POST	/api/vetcare/consultations	201, 400, 404
POST	/api/vetcare/vaccinations	201, 400, 404
POST	/api/vetcare/treatments	201, 400, 404
GET	/api/vetcare/animals/{id}/history	200, 404

Interactions avec les autres microservices :



1. Adoption consulte le dossier médical et le statut vaccinal d'un animal auprès de VetCare (REST)
2. Animal secouru à orienter vers un vétérinaire : événement RescueMissionCompleted de Animal Rescue
3. Si un service est en panne : le message reste dans la file jusqu'à son retour

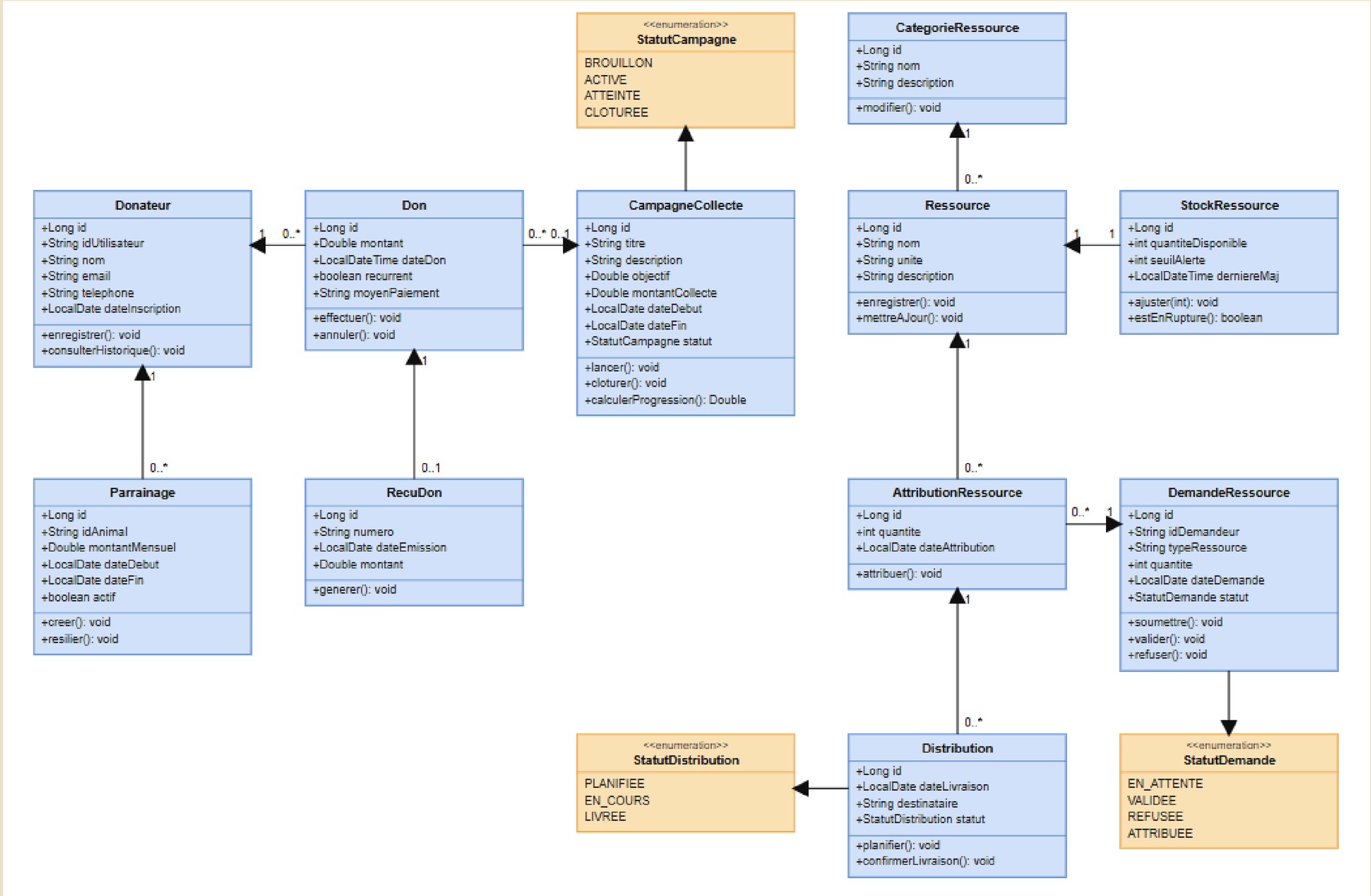
Architecture de microservice :



Micro Service VI:

Donation & Ressource Management

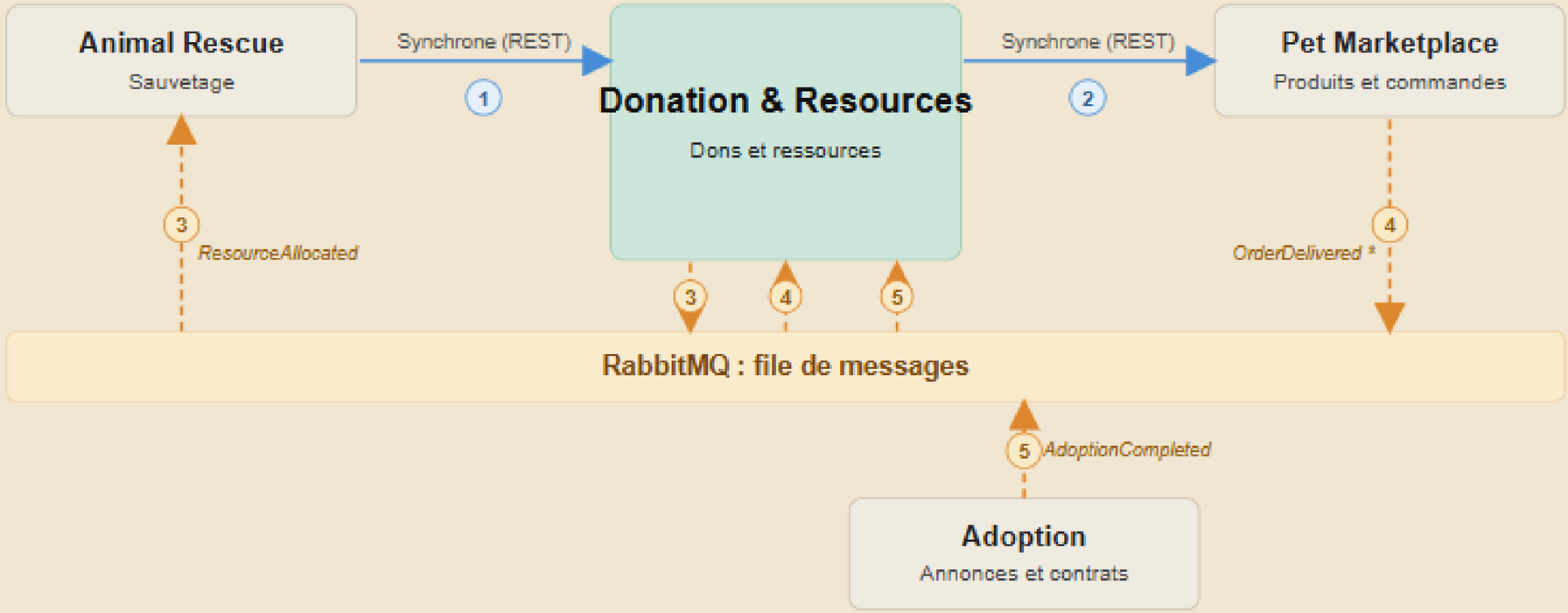
Diagramme de classe :



II API REST du Donation & Resources:

Méthode	Endpoint	Codes de réponse
POST	/api/donations/campaigns	201, 400
GET	/api/donations/campaigns	200
POST	/api/donations	201, 400, 404
GET	/api/donations/history	200
POST	/api/donations/sponsorships	201, 400, 404
POST	/api/resources	201, 400
GET	/api/resources/inventory	200
POST	/api/resources/requests	201, 400
PUT	/api/resources/requests/{id}/decision	200, 400, 404, 409
POST	/api/resources/allocations	201, 400, 404, 409
GET	/api/resources/distributions	200

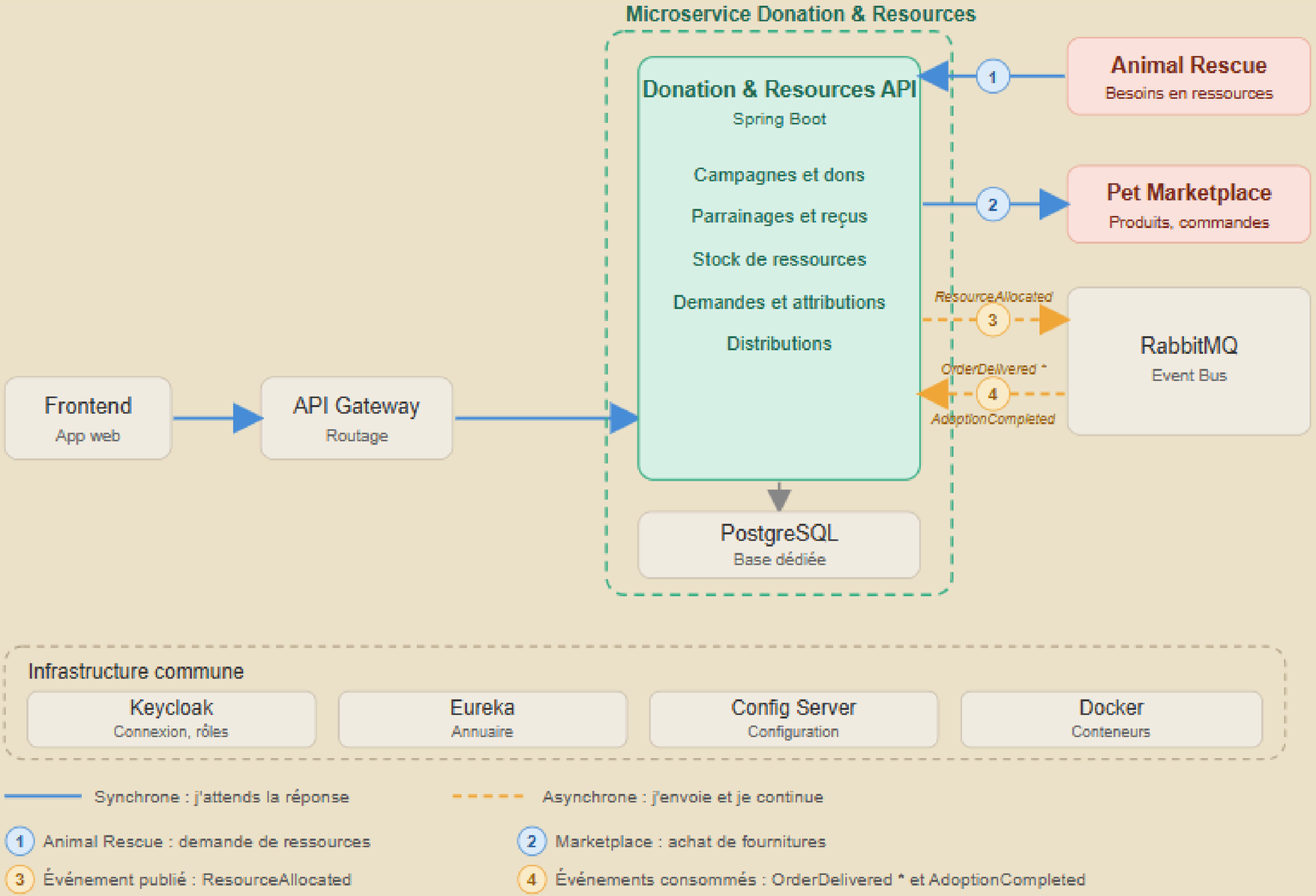
Interactions avec les autres microservices :



1. Animal Rescue demande nourriture, médicaments et équipements à Donation & Resources (REST)
2. Donation & Resources achète à Pet Marketplace les fournitures manquantes au stock (REST)
3. Donation & Resources publie ResourceAllocated sur RabbitMQ : Animal Rescue confirme les fournitures pour son opération
4. Pet Marketplace publie OrderDelivered * (commande livrée) : Donation & Resources met à jour son stock de ressources
5. Adoption publie AdoptionCompleted : Donation & Resources met fin au parrainage de l'animal

** Événement proposé par le responsable du Marketplace, à valider avec l'équipe.*

Architecture de microservice :



Planning prévisionnel et risques

Phase 1 Conception

Architecture, diagrammes, dépôt Git, présentation

Phase 2 Infrastructure

Gateway, Eureka, Config Server, Keycloak, Docker Compose

Phase 3 Développement

Workflows essentiels de chaque microservice et service collectif

Phase 4 Intégration

RabbitMQ, frontend, tests de bout en bout, démo

PRINCIPAUX RISQUES ET RÉPONSES

Service collectif à cinq

Conflits de code : une partie par membre, branches par fonctionnalité, revue des pull requests.

Contrats d'API instables

RabbitMQ, frontend, tests de bout en bout, démo

Infrastructure complexe

Keycloak, Eureka, Gateway : socle posé tôt, Membre 5 en appui à l'intégration.

Panne ou retard d'un service

RabbitMQ garde les événements en file ; suivi des tâches et point hebdomadaire.

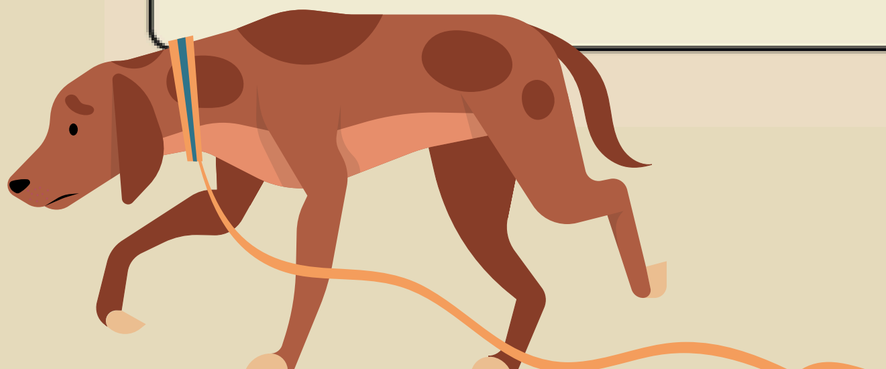
Conclusion:

Ce que nous avons conçu

- **6 microservices indépendants**
  5 services individuels et 1 service collectif, chacun avec sa base PostgreSQL et un périmètre clair.
- **Des échanges définis**
  REST pour les réponses immédiates, RabbitMQ pour les événements entre services.
- **Un projet organisé**
  Dépôt Git structuré, un responsable par service, tâches réparties dans l'équipe.

Prochaines étapes

- **Infrastructure**
  Docker Compose, bases PostgreSQL, RabbitMQ, API Gateway.
- **Développement**
  Workflows essentiels de chaque service, puis le service collectif.
- **Intégration et tests**
  Appels REST, événements, scénarios de bout en bout, application web.



Woof!

Thank you !

